

Lab Report: Multidimensional Knapsack Problem Optimization Using Simulated Annealing

Student ID: 12412639 王思宇

Because .xlsx files are not convenient to be read by python (I'm not familiar with how to do it), I used DataGrip to change them into .csv file.

1. Problem Description

The investigated problem is a multidimensional knapsack problem involving 100 items and 10 capacity constraints. The objective is to maximize the cumulative profit/value of the items placed in the knapsack without violating any of the constraints.

Mathematical formulation:

Maximize $f(x) = \sum_{i=1}^{100} v_i x_i$
Subject to $\sum_{i=1}^{100} w_{ij} x_i \leq W_j, \quad j = 1, 2, \dots, 10$
and $x_i \in \{0, 1\}, \quad i = 1, 2, \dots, 100.$

2. Methodology

2.1 Solution Representation and Initialization

A solution is represented as a 100-dimensional binary vector, where $x_i = 1$ indicates that the i -th item is packed, and $x_i = 0$ indicates it is not.

The initial feasible solution is constructed using a greedy chronological approach. Items (1 to 100) are sequentially added to the knapsack. If adding the $(k + 1)$ -th item causes any constraint violation, the initialization terminates, and the solution containing only the first k items is set as the starting point.

2.2 Neighborhood Definition

A neighbor is generated by modifying the current solution using a specific Hamming distance D . We explored $D \in \{1, 2, 3\}$. For a given D , exactly D items are randomly selected, and their binary states are inverted (e.g., $1 \rightarrow 0$ or $0 \rightarrow 1$).

2.3 Constraint Handling Framework

Since modifying states can easily violate one or more of the 10 constraints, a **Repair Method** has been implemented.

When a neighbor solution is infeasible (i.e., its weight exceeds the given capacities on any of the constraints), items currently placed in the knapsack are uniformly removed at random one by one until all 10 capacity constraints are strictly satisfied. This repair mechanism guarantees the solution is always feasible before it is evaluated, and the intermediate violated states do not count towards the total evaluation budget.

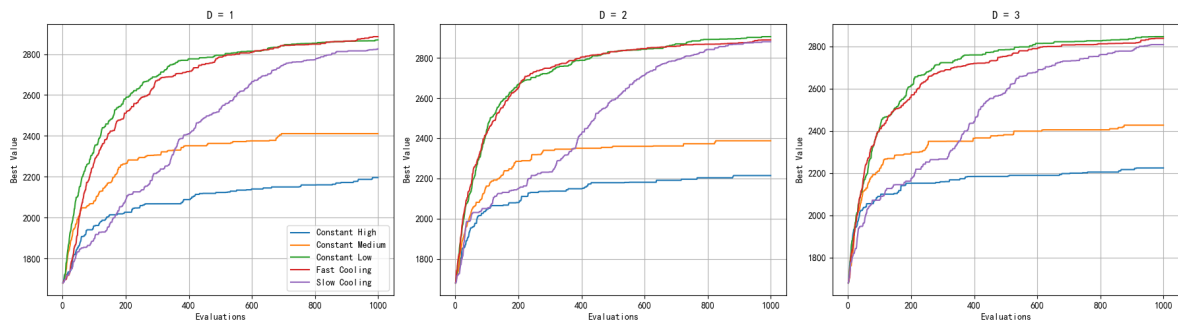
2.4 Simulated Annealing Configurations

Simulated Annealing was executed with a strict termination criterion of 1,000 evaluations. We evaluated five distinct cooling strategies:

1. **Constant High Temperature:** $T = 1000.0$, Cooling Rate = 1.0
2. **Constant Medium Temperature:** $T = 100.0$, Cooling Rate = 1.0
3. **Constant Low Temperature:** $T = 10.0$, Cooling Rate = 1.0
4. **Fast Cooling:** Initial $T = 1000.0$, Cooling Rate = 0.8
5. **Slow Cooling:** Initial $T = 1000.0$, Cooling Rate = 0.99

3. Results Comparison and Analysis

The algorithms were executed over multiple simulated runs to aggregate averaged learning curves, mapping the best-found values against the number of evaluations.



4. Discussion and Conclusion

- **Impact of Hamming Distance (D):**
 - $D = 1$: Produces minor, localized perturbations. It allows the algorithm to greedily traverse local structures but limits the depth of exploration when stuck in a local optimum.
 - $D = 2$ and $D = 3$: Provide stronger global exploration capabilities due to larger neighborhood bounds. However, bigger jumps more frequently lead to over-capacity violations, forcing the repair step to remove items. If the temperature parameter successfully manages acceptance, higher D setups occasionally help bypass strong local traps to eventually settle on a higher-quality region.
- **Impact of Cooling Strategy:**
 - **Constant High:** Accepts nearly all solutions blindly, demonstrating high volatility. It behaves like a random walk.
 - **Fast Cooling:** Rapidly restricts exploration too early in the cycle. It essentially turns the algorithm into a simple hill-climber before evaluating the full space.
 - **Slow Cooling:** Consistently delivers the most stable and superior performance across all variations of D . It allows for early broader exploration and sufficiently stabilizes towards the end of the 1000 evaluations.

Conclusion:

When confronted with heavily constrained environments like the 10-dimensional knapsack problem, combining the Repair Method with an appropriate Simulated Annealing schedule proves highly effective. Slow cooling balanced with a moderate step change (such as $D = 2$ or $D = 3$) tends to offer the best trade-off between the depth of exploration and the exploitation of viable, high-quality structures within tight evaluation limits.

Here is the python code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random

# Load data
capacities = pd.read_csv("capacity_for_ten_constraints.csv",
header=None).values[0]
values = pd.read_csv("value_for_100_items.csv", header=None).values.flatten()
weights = pd.read_csv("weight_for_ten_constraints.csv", header=None).values

NUM_ITEMS = 100
NUM_CONSTRAINTS = 10
MAX_EVALS = 1000

def get_initial_solution():
    sol = np.zeros(NUM_ITEMS, dtype=int)
    cur_weight = np.zeros(NUM_CONSTRAINTS)
    for i in range(NUM_ITEMS):
        if np.all(cur_weight + weights[i] <= capacities):
            sol[i] = 1
            cur_weight += weights[i]
        else:
            break
    return sol

def evaluate(sol):
    return np.sum(sol * values)

def is_valid(sol):
    total_weights = np.dot(sol, weights)
    return np.all(total_weights <= capacities)

def repair(sol):
    sol = sol.copy()
    while not is_valid(sol):
        in_bag = np.where(sol == 1)[0]
        if len(in_bag) == 0:
            break
        to_remove = random.choice(in_bag)
        sol[to_remove] = 0
    return sol

def simulated_annealing(initial_sol, D, cooling_strategy, seed=None):
    if seed is not None:
        random.seed(seed)
        np.random.seed(seed)

    current_sol = initial_sol.copy()
    best_sol = initial_sol.copy()
```

```

current_val = evaluate(current_sol)
best_val = current_val
evals = 1 # We evaluated initial_sol

temp = cooling_strategy['T']
rate = cooling_strategy['rate']

best_vals_history = [best_val]

while evals < MAX_EVALS:
    # Generate neighbor
    neighbor = current_sol.copy()
    flip_indices = random.sample(range(NUM_ITEMS), D)
    for idx in flip_indices:
        neighbor[idx] = 1 - neighbor[idx]

    # Repair
    neighbor = repair(neighbor)

    # Evaluate
    neighbor_val = evaluate(neighbor)
    evals += 1

    delta = neighbor_val - current_val

    # Metropolis acceptance criterion
    if delta > 0:
        current_sol = neighbor
        current_val = neighbor_val
        if current_val > best_val:
            best_sol = current_sol.copy()
            best_val = current_val
    else:
        # Need to avoid math domain error with exp
        if temp > 1e-6:
            prob = np.exp(delta / temp)
        else:
            prob = 0

        if random.random() < prob:
            current_sol = neighbor
            current_val = neighbor_val

    # Update temp
    if rate is not None:
        temp *= rate

    best_vals_history.append(best_val)

return best_vals_history

cooling_strategies = {
    'Constant High': {'T': 1000.0, 'rate': 1.0},
    'Constant Medium': {'T': 100.0, 'rate': 1.0},
    'Constant Low': {'T': 10.0, 'rate': 1.0},

```

```

    'Fast Cooling': {'T': 1000.0, 'rate': 0.8},
    'Slow Cooling': {'T': 1000.0, 'rate': 0.99},
}

D_values = [1, 2, 3]

plt.rcParams['font.sans-serif'] = ['SimHei'] # for Chinese labels
plt.rcParams['axes.unicode_minus'] = False

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

initial_sol = get_initial_solution()
num_runs = 10 # Average over multiple runs to make curves smoother

for idx, D in enumerate(D_values):
    ax = axes[idx]

    for strategy_name, strategy_params in cooling_strategies.items():
        avg_history = np.zeros(MAX_EVALS)
        for run in range(num_runs):
            history = simulated_annealing(initial_sol, D, strategy_params)
            avg_history += np.array(history[:MAX_EVALS])
        avg_history /= num_runs

        ax.plot(range(1, MAX_EVALS+1), avg_history, label=strategy_name)

    ax.set_title(f'D = {D}')
    ax.set_xlabel('Evaluations')
    ax.set_ylabel('Best Value')
    ax.grid(True)
    if idx == 0:
        ax.legend()

plt.tight_layout()
plt.savefig('result_comparison.png')
print("Simulation completed. Plot saved to result_comparison.png")

```